



檢索系統設計

System Design

SENA 2A

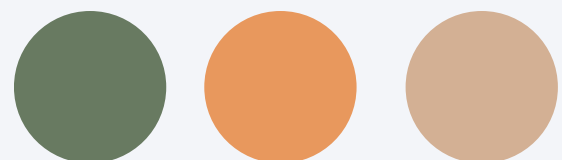
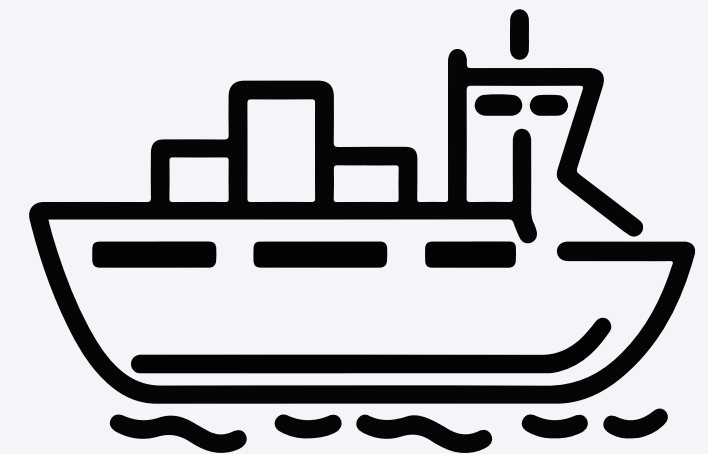
01351037 張立醒

01351011 許紘境

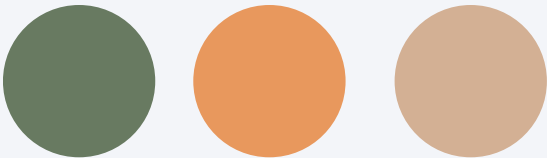
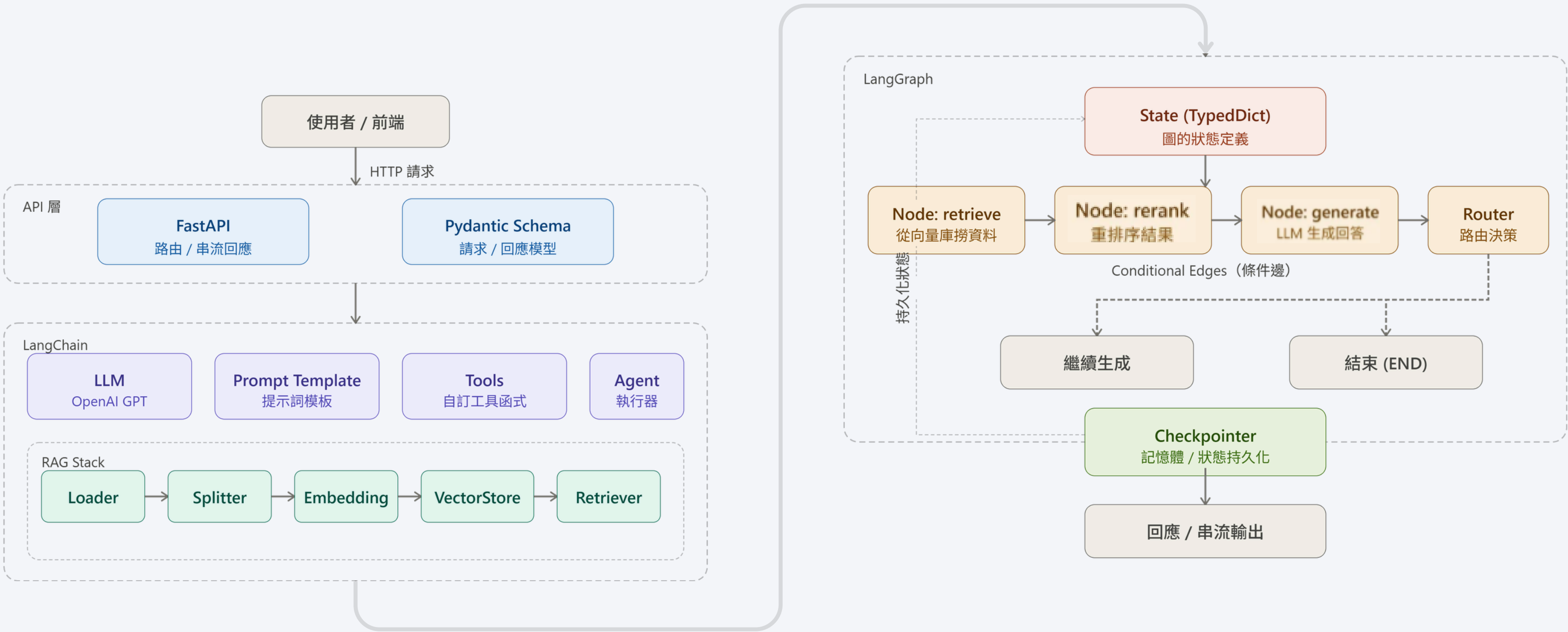
01351018 李宇陽

OUTLINE

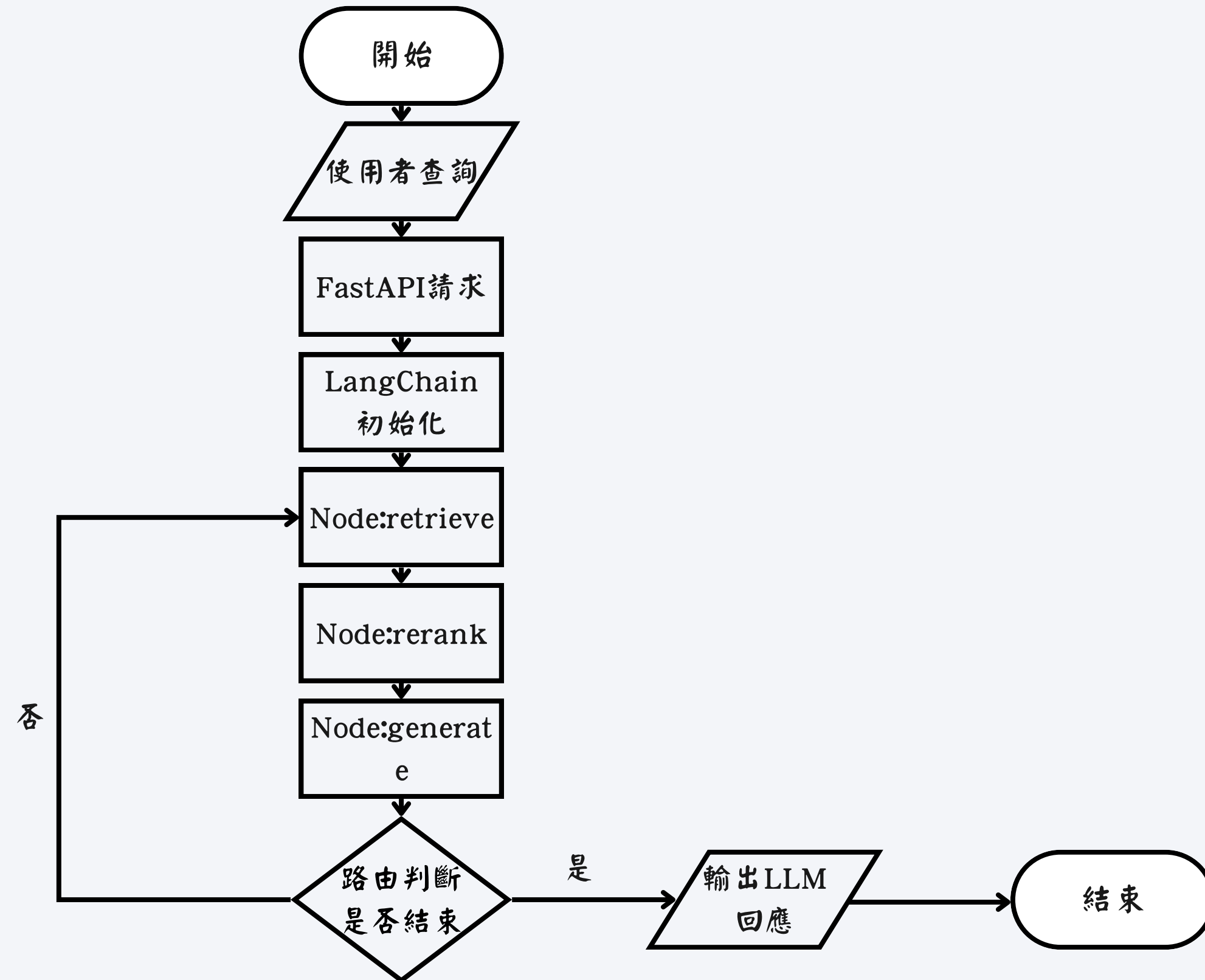
- 前端網站(Frontend UI)
- 後端伺服器(Backend Server & Agent Workflow)
- RAG 檢索核心
- 資料庫與API(Database & LLM)

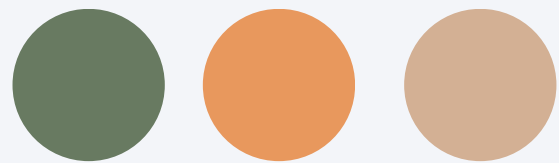


系統架構圖



流程圖





前端網站(Frontend UI)



輸入 & 輸出介面

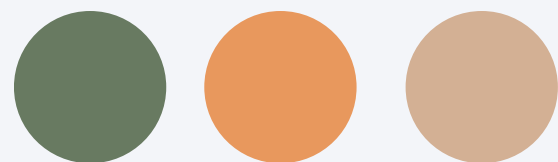
- 支援中英文輸入框，以及圖片和PDF上傳
(圖片以及PDF用於船體圖or計算公式)
- 對話是呈現回覆，並具備「資料來源引用」，讓搜尋者可以查找原始的規範或圖表

技術類型

- Vue.js 3(Composition API + Vite)

核心職責

- 使用者互動介面、多模態檔案上傳、即時串流回覆響應



什麼是Vue.js？

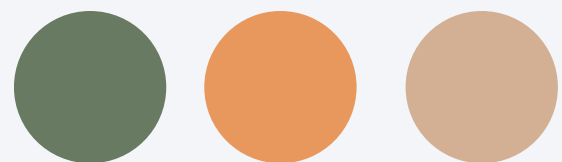
核心觀念

- 響應式系統（Reactivity）
- 雙向資料綁定（Two-way Binding）
- 與組件化架構（Components）。

在傳統網頁開發中，當資料改變時，我們必須手動尋找 HTML 元素並更新它；而在 Vue.js 中，只需要管理資料，畫面會像 Excel 公式一樣自動同步更新。



範例



什麼是Vue.js ?

左半邊 (畫面的大腦 - Model) :

```
const newTask = ref('');  
// 監視這個變數  
const addTask = () => {  
  // 執行新增動作  
  
  tasks.value.unshift(newTask.value);  
}
```

右半邊 (使用者的畫面 - View) :

```
<!-- 雙向綁定變數 -->  
<input v-model="newTask">  
  <!-- 綁定點擊事件 -->  
<button @click="addTask">新增</button>
```



什麼是Vue.js ?

造船工程任務清單

Vue 3 組合式 API (Composition API) 示範

新增

☒ 閱讀 ABS 鋼船建造規範

✕

☐ 計算船體基礎阻力係數

✕

☐ 設計知識代理工作流

✕

總計：3 項任務

已完成：1 / 3



```
const completedCount =  
  computed(() => {  
    return tasks.value.filter(task  
      => task.done).length;  
  });
```

範例



Vue.js對於此系統的優點？

- 完美契合 AI 串流對話與多模態輸入

當後端AI代理正在檢索龐大的IMO規範時，文字會一個字一個字傳回前端。

透過 Vue 的雙向綁定，我們可以零延遲在畫面上呈現出使用者打的字，且在使用者上傳船體圖表的同時，畫面完全不會卡頓。

- 動態法規引用過濾器

造船設計非常嚴謹，AI 每一句話都必須附帶文獻來源。

我們可以利用 Vue 的計算屬性（computed），讓它在背景自動監視 AI 的回答。

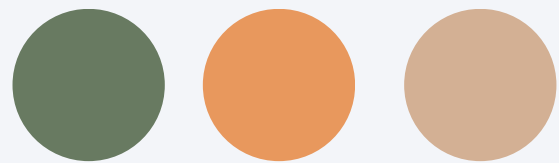
只要 AI 一吐出文獻標籤，Vue就會瞬間把它整理成側邊欄的『參考文獻清單』

- 即時渲染複雜的工程公式

系統生成的流體力學公式（如船舶阻力計算）通常是生硬的 LaTeX代碼。

有了Vue，當在輸入框調整船長或速度參數時，旁邊預覽區的公式與計算結果就能像智慧試算表一樣瞬間更新。





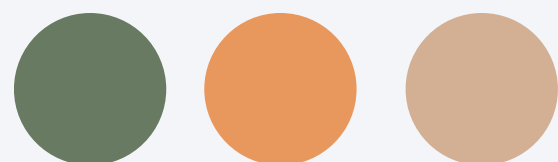
後端伺服器(Backend Server & Agent Workflow)



核心框架

本系統流程設計參考 LangChain／LangGraph

- 基於Python建立後端，並透過LangChain以及LangGraph構建代理工作流
- LangChain：用於整合模型、提示詞、工具與 Agent
 - a. 初始化語言模型 (Large Language Model)
 - b. 建立提示詞模板 (Prompt Template)
 - c. 定義工具 (Tools)
 - d. 實例化代理與執行器 (Agent & AgentExecutor)
- LangGraph：流程節點與狀態管理概念
 - a. 明確定義路由
 - b. 記憶體管理 (Checkpointing)



LangChain 核心模組與代理基礎

- LLM (大語言模型)：

採用 OpenAI GPT 作為系統的「大腦」，負責語意理解與自然語言生成。

- Prompt Template (提示詞模板)：

改寫使用者的輸入格式，提供上下文引導，確保 LLM 產出符合專業領域要求的回應。

- Tools (自訂工具函式)：

賦予 AI 突破文字限制的能力，允許系統動態呼叫外部資源（如：工程公式計算）。

- Agent (執行器)：

擔任任務指揮官，負責拆解使用者的複雜問題，並自主決定何時、按何種順序調用 Tools 來完成任務。



LangGraph 動態 workflow

State 全域狀態管理 (TypedDict):

- 定義嚴謹的狀態字典，全程追蹤使用者的查詢意圖、中繼檢索資料與對話歷史，確保流程上下文不遺失。

路由與決策:

- 接收用戶查詢後，由 LLM 判斷這是一個純文字知識題、法規查核題，還是圖片/公式解析題，並分發到對應的處理管線。
- 若資訊不足或需進一步推理，觸發迴圈重新處理。
- 若判定回答已完整且準確，則終止 workflow 並向前端輸出最終結果。

動態執行節點 (Execution Nodes):

- Node: retrieve (檢索資料): 對接資料庫，精準撈取相關的文本或數據。
- Node: rerank (重排序): 針對撈取或生成的結果進行二次評估與重排序，過濾雜訊以提升精準度。
- Node: generate (生成回答): 統整上下文與檢索資訊，驅動 LLM 進行邏輯推理並生成回答。



LangGraph 動態 workflow

Checkpoint:

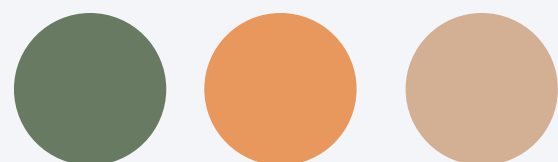
- Checkpointer 的工作，就是在每一個節點（Node）執行完畢的瞬間，把這個 State 寫進資料庫裡，方便未來調取與接續中斷動作。

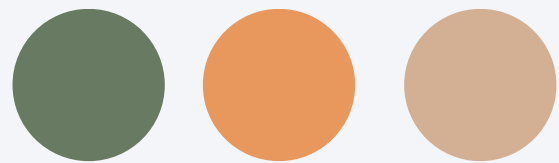
讓 AI 擁有「記憶」（Memory）:

- Checkpointer 能夠記住多回合的對話與圖紙參數，AI 才能理解代稱詞(那、這、那些)是指什麼，順暢地進行多輪對話。

斷點續傳:

- 如果 Agent 走到 Node: generate 時，突然遇到 OpenAI 伺服器斷線或超時，系統不會整個崩潰。因為 Checkpointer 會把前面 Node: retrieve（檢索資料）的結果存下來了。下次重試時，系統可以直接從生成節點繼續，不用重新去向量庫撈一次資料。

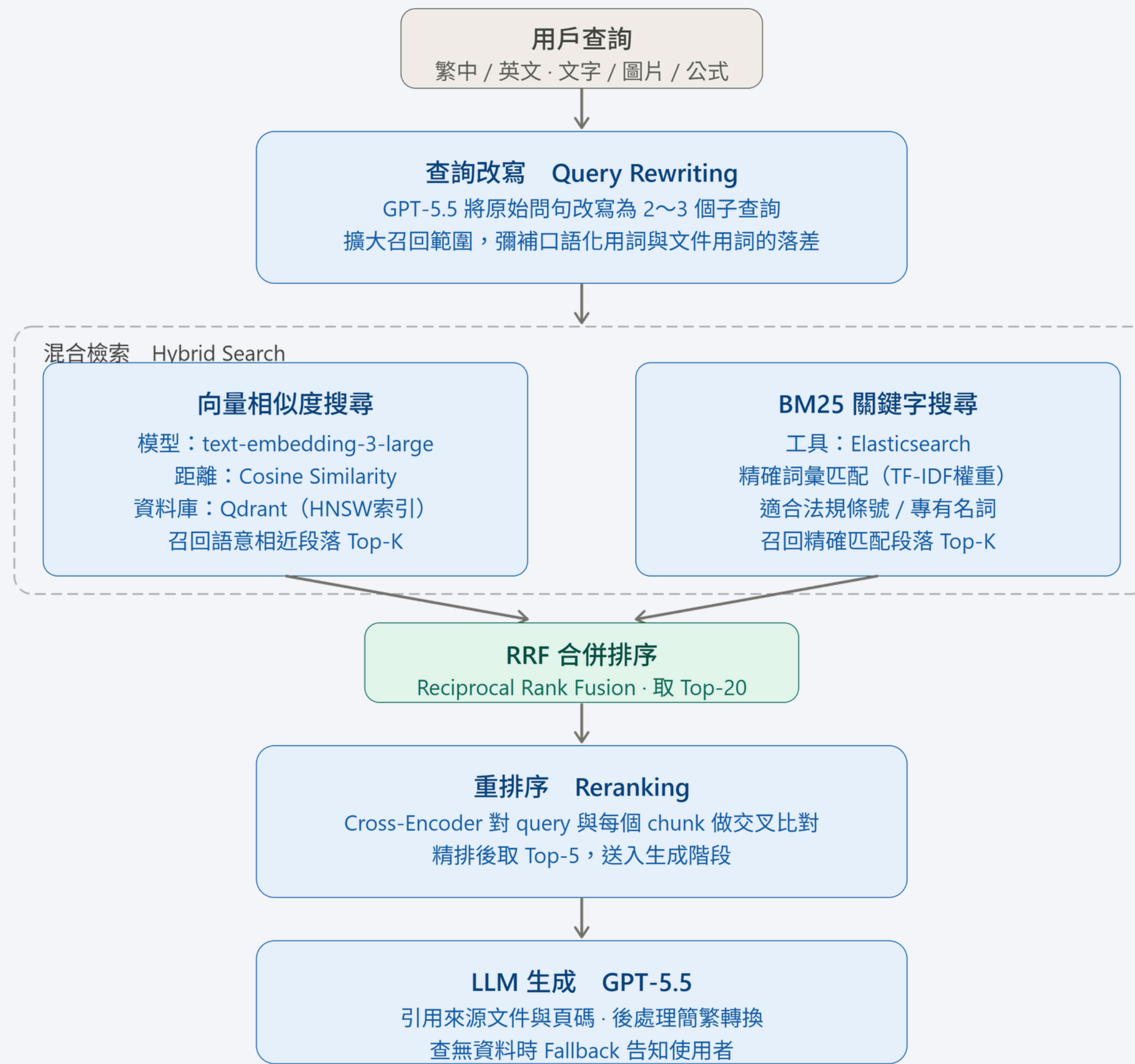
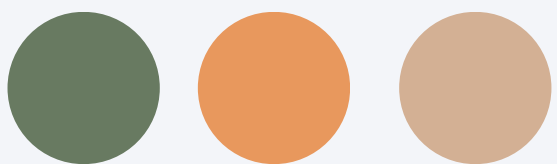




RAG 檢索核心



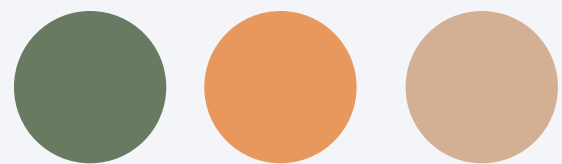
RAG（檢索增強生成，Retrieval-Augmented Generation）是一種讓 AI 在回答問題前，先到外部知識庫搜尋相關資料，再根據這些資料來生成回答的技術。



核心架構

- 文件前處理
PDF 解析 / 公式圖片抽取 /
Chunk 切割 / Metadata
- Embedding 向量化
OpenAI text-embedding-3-large
/ 向量資料庫

- 混合檢索 Hybrid Search
向量搜尋 + BM25 + RRF 合併
排序
- 重排序與查詢改寫
Cross-Encoder Reranker /
Multi-Query 擴增

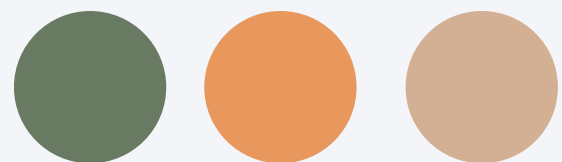


整體 RAG 流程



[後處理] 偵測並自動轉換所有簡體字為繁體中文

[Fallback] 資料庫查無時改用 GPT-5.5 自身知識，並明確告知使用者



文件前處理與切割策略

PDF 解析

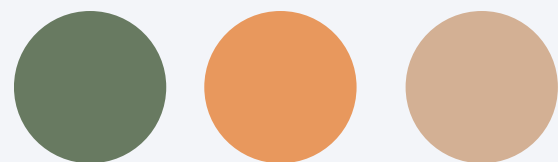
- 使用 PyMuPDF 逐頁解析文字與圖片
- 偵測並抽取嵌入公式 (LaTeX/MathML)
- 圖片另存為獨立檔案供多模態 Embedding
- 保留原始頁碼結構供溯源使用

Chunk 切割策略

- Recursive Text Splitter
- chunk size : 512 token
- overlap : 64 token
- 優先在段落、句號邊界切割，保留語意完整

Metadata 設計

- 文件名稱 / 頁碼 / 段落編號
- 內容類型：文字 / 圖片 / 公式
- 來源語言：繁中 / 英文
- 版本日期 (Rules 2026)



Embedding 向量化與儲存

Embedding 模型

主力模型：text-embedding-3-large (OpenAI)

向量維度：3072 dim

語言支援：繁中、英文 (本系統雙語文件)

公式處理：LaTeX 字串轉文字後向量化

圖片處理：GPT-5.5 Vision 生成說明文字後向量化

向量資料庫設計

資料庫：Qdrant (或pgvector)

索引方式：HNSW — 高效近似最近鄰搜尋

距離計算：Cosine Similarity

Collection：依文件分集合，支援跨集合查詢

版本控制：文件更新時自動重新向量化



混合檢索 Hybrid Search

用戶查詢

向量相似度搜尋

語意層

text-embedding-3-large

Cosine Similarity

Top-K 召回

BM25 關鍵字搜尋

詞彙層

Elasticsearch/Typesense

精確詞彙匹配

法規條號・專有名詞

RRF 合併排序
Reciprocal Rank Fusion

為什麼用向量搜尋？

將查詢與文件轉為語意向量，即使用詞不同也能找到語意相近的段落，適合處理自然語言問句。

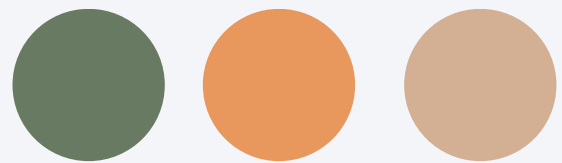
為什麼同時用 BM25？

工程規範中大量使用條號（Art.3.1.2）、材料代號等精確詞彙，BM25 對完全匹配的表現遠勝向量搜尋。

為什麼用 RRF 合併？

RRF 以排名倒數和取代分數直接加總，避免兩種分數量綱不同的問題，是業界成熟的融合策略。





資料庫與API(Database & LLM)



Database系統資料儲存

對話狀態儲存：

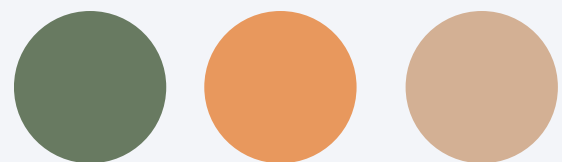
記憶體狀態持久化：負責LangGraph的 Checkpointer 機制，建立關聯式資料庫（如MySQL），將使用者的多輪對話歷史存下來，讓系統有記憶。

系統日誌紀錄：紀錄每次 API 呼叫的狀態與錯誤日誌，維持伺服器穩定運作。

原始檔案儲存：

存放圖紙與文件：建立一個存放空間（如本地儲存或 AWS S3），用來安全地儲存前端上傳的 PDF 文件、船體圖以及計算公式圖片。

支援資料溯源：提供檔案下載或預覽的 API URL，這樣當前端需要顯示「資料來源引用」時，才能抓到最原始的圖表或規範檔案。



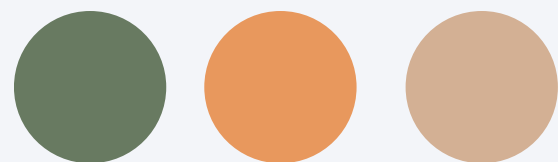
API 服務與通訊架構FastAPI + Pydantic

作為整個系統的通訊樞紐，定義標準化的資料交換格式。

資料模型定義 (Schema)：使用 Pydantic 嚴格定義前端請求（如：用戶提問、上傳檔案）與後端回應（如：文字、引用來源）的資料結構，確保前後端對接不打結。

非同步與串流支援 (Streaming)：規劃支援 Server-Sent Events (SSE) 的串流輸出架構，讓前端能即時顯示 AI 逐字生成的回答。

*Pydantic:是 Python 的一個資料驗證庫，結合 FastAPI 使用時威力強大。可以預先寫好一個 Schema（資料模型藍圖），規定好傳進來的資料「必須」長什麼樣子，如果前端傳來的資料不符合這個嚴格的定義，Pydantic 連讓這筆資料進入系統都不肯，會直接在門口擋下，並自動回傳一個非常明確的 HTTP 422 錯誤訊息告訴前端「你哪個欄位傳錯了」。



模型與外部工具閘道 (LLM & Tools Gateway)

環境變數安全控管 + 統一工具封裝介面。

模型串接與控管：統一管理外部 LLM (如 OpenAI GPT-4o) 的 API 連線、金鑰安全、Timeout 機制與自動重試邏輯。

自訂工具 API 封裝 (Tools)：將專案所需的外部專業計算程式 (如船舶工程計算、流體力學運算) 或外部法規查詢介面，封裝成標準的 API 函式，提供給後端 Agent 模組進行註冊與呼叫。

LLM控制：

給予LLM限制，避免輸出非資料集的資料，以及不可顯示簡體中文等。

*除了讓使用AI本身限制外，也可在AI生成回答後再用程式碼去過濾簡體字，這樣能夠更保證系統的回覆符合設定與規範。



参考文献

Vue.js : <https://vuejs.org/guide/introduction.html>

LangGraph : <https://docs.langchain.com/oss/python/langgraph/overview>

LangChain : <https://docs.langchain.com/oss/python/langchain/overview>

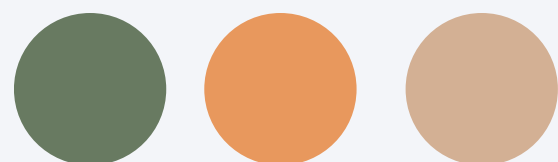
Retrieval-Augmented Generation(RAG) : <https://arxiv.org/abs/2005.11401>

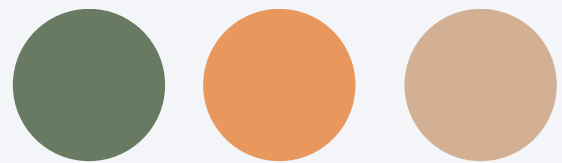
PyMuPDF : <https://pymupdf.readthedocs.io/en/latest/>

Embedding : <https://developers.openai.com/api/docs/guides/embeddings>

Fast API : <https://fastapi.tiangolo.com/>

Pydantic : <https://pydantic.dev/docs/validation/latest/get-started/>





Thank You!

